

```

1  import os
2  import sys
3  import ctypes
4  import tkinter as tk
5  from tkinter import font as tkfont
6  from pylogix import PLC
7
8  #=====
9  # ENGIE OPERAÇÃO E MANUTENÇÃO
10 # PROGRAMADO POR GEOVANE FERRAZ MONTEIRO, 18/05/2026
11 # PROIBIDA A DISTRIBUIÇÃO SEM AVISO PRÉVIO E CONCENTIMENTO
12 # =====
13 # FUNÇÃO PARA CARREGAR FONTE EM TEMPO DE EXECUÇÃO (SEM INSTALAR NO WINDOWS)
14 # =====
15 def carregar_fonte_local(caminho_fonte):
16     """Carrega temporariamente um arquivo .ttf na memória do Windows para o script
17     usar."""
18     if sys.platform == "win32":
19         if os.path.exists(caminho_fonte):
20             FR_PRIVATE = 0x10
21             retorno = ctypes.windll.gdi32.AddFontResourceExW(caminho_fonte, FR_PRIVATE, 0
22             )
23             if retorno != 0:
24                 print(f"Font carregada com sucesso a partir de: {caminho_fonte}")
25                 return True
26             else:
27                 print("Falha ao registrar a fonte via API do Windows.")
28         else:
29             print(f"Aviso: Arquivo de fonte '{caminho_fonte}' não foi encontrado na
30             pasta.")
31     else:
32         print("Carregamento dinâmico automático configurado apenas para Windows.")
33     return False
34
35 # =====
36 # CONFIGURAÇÕES DO CLP E ACESSO
37 # =====
38 IP_CLP = '172.16.35.30'
39 SLOT_CLP = 2
40
41 TAGS_PARA_LER = [
42     'relogio[3]', # Hora
43     'relogio[4]', # Minuto
44     'relogio[5]', # Segundo
45     'relogio[2]', # Dia
46     'relogio[1]', # Mês
47     'relogio[0]', # Ano
48     'Conta[0].acc',
49     'Conta[1].acc',
50     'Reais[0]',
51     'Reais[1]',
52     'Reais[2]',
53     'Reais[3]',
54     'Reais[4]',
55     'Reais[5]'
56 ]
57
58 INTERVALO_ATUALIZACAO = 100 # Tempo em milissegundos (100ms = 10Hz)
59
60 ARQUIVO_FONTE = "LCDAT&TPhoneTimeDate.ttf"
61 NOME_INTERNO_FONTE = "LCD AT&T Phone Time/Date"
62
63 carregar_fonte_local(ARQUIVO_FONTE)
64
65 # =====
66 # CLASSE PRINCIPAL DA CLASSE VISUAL (GUI)
67 # =====
68 class MonitorCLPApp:

```

```

65 def __init__(self, root):
66     self.root = root
67     self.root.title("Supervisório CLP - pylogix")
68     self.root.geometry("750x850")
69     self.root.configure(bg="#1e1e1e")
70
71     # -----
72     # SELEÇÃO DE FONTES CUSTOMIZADAS
73     # -----
74     self.fonte_titulo = tkfont.Font(family="Consolas", size=16, weight="bold")
75     self.fonte_cabecalho = tkfont.Font(family="Consolas", size=11, weight="bold")
76     self.fonte_dados = tkfont.Font(family="Consolas", size=11, weight="normal")
77     self.fonte_lcd_grande = tkfont.Font(family=NOME_INTERNO_FONTE, size=48, weight=
"bold")
78     self.fonte_lcd_media = tkfont.Font(family=NOME_INTERNO_FONTE, size=20, weight=
"bold")
79
80     # Inicializa o driver do pylogix
81     self.comm = PLC()
82     self.comm.IPAddress = IP_CLP
83     self.comm.ProcessorSlot = SLOT_CLP
84
85     # Variáveis de controle para o Indicador (Heartbeat) e Histórico
86     self.heartbeat_state = False
87     self.historico_conta0 = [] # Armazena os últimos pontos da tag Conta[0].acc
88     self.historico_segundos = [] # Armazena os últimos pontos da tag relógio[5]
(Segundos)
89     self.max_pontos_grafico = 50 # Quantidade máxima de pontos na tela
90
91     self.widgets_dados = {}
92     self.criar_layout()
93     self.root.bind("<Escape>", self.fechar_aplicativo)
94     self.atualizar_dados()
95
96 def criar_layout(self):
97     """Monta a estrutura visual da tabela, relógio, indicador e gráfico."""
98     topo_frame = tk.Frame(self.root, bg="#1e1e1e")
99     topo_frame.pack(padx=20, pady=10, fill=tk.X)
100
101     # --- LINHA 1: TÍTULO PRINCIPAL ---
102     linha1_frame = tk.Frame(topo_frame, bg="#1e1e1e")
103     linha1_frame.pack(fill=tk.X, side=tk.TOP)
104
105     lbl_titulo = tk.Label(
106         linha1_frame,
107         text="MONITORAMENTO EM TEMPO REAL",
108         font=self.fonte_titulo, fg="#00ff66", bg="#1e1e1e"
109     )
110     lbl_titulo.pack(side=tk.LEFT, pady=(0, 5))
111
112     # --- LINHA 2: IP DO CLP E STATUS DA COMUNICAÇÃO ---
113     linha2_frame = tk.Frame(topo_frame, bg="#1e1e1e")
114     linha2_frame.pack(fill=tk.X, side=tk.TOP, pady=(5, 0))
115
116     lbl_ip = tk.Label(
117         linha2_frame,
118         text=f"CLP IP: {IP_CLP}",
119         font=self.fonte_cabecalho, fg="#9cdcfe", bg="#1e1e1e"
120     )
121     lbl_ip.pack(side=tk.LEFT)
122
123     self.canvas_led = tk.Canvas(linha2_frame, width=25, height=25, bg="#1e1e1e", bd=0
, highlightthickness=0)
124     self.canvas_led.pack(side=tk.RIGHT, padx=(5, 0))
125     self.led_id = self.canvas_led.create_oval(3, 3, 22, 22, fill="#444444", outline=
"#666666")
126

```

```

127 lbl_led_texto = tk.Label(linha2_frame, text="COM:", font=self.fonte_cabecalho, fg
128  ="#ffffff", bg="#1e1e1e")
129
130 # -----
131 # PAINEL DO RELÓGIO DIGITAL UNIFICADO
132 # -----
133 frame_relogio = tk.LabelFrame(self.root, text=" HORÁRIO DO CLP ", font=self.
134   fonte_cabecalho, fg="#ffffff", bg="#151515", bd=2, relief=tk.GROOVE)
135 frame_relogio.pack(padx=20, pady=5, fill=tk.X)
136
137 self.lbl_relogio_hora = tk.Label(frame_relogio, text="--:--:--", font=self.
138   fonte_lcd_grande, fg="#00ffff", bg="#151515")
139 self.lbl_relogio_hora.pack(pady=2)
140
141 self.lbl_relogio_data = tk.Label(frame_relogio, text="DATA: --/--/----", font=
142   self.fonte_dados, fg="#888888", bg="#151515")
143 self.lbl_relogio_data.pack(pady=2)
144
145 # -----
146 # PAINEL DAS VARIÁVEIS (TABELA DINÂMICA)
147 # -----
148 frame_tabela = tk.LabelFrame(self.root, text=" VARIÁVEIS DE PROCESSO ", font=self.
149   fonte_cabecalho, fg="#ffffff", bg="#1e1e1e", bd=0)
150 frame_tabela.pack(padx=20, pady=5, fill=tk.X)
151
152 frame_tabela.columnconfigure(0, weight=2)
153 frame_tabela.columnconfigure(1, weight=1)
154 frame_tabela.columnconfigure(2, weight=1)
155
156 cabecalhos = ["TagName", "Value", "Status"]
157 for col_idx, texto in enumerate(cabecalhos):
158     lbl = tk.Label(
159         frame_tabela, text=texto, font=self.fonte_cabecalho,
160         anchor="center", fg="#ffffff", bg="#2d2d2d", relief=tk.FLAT
161     )
162     lbl.grid(row=0, column=col_idx, sticky="nsew", padx=1, pady=2, ipady=4)
163
164 tags_exibicao = ['Conta[0].acc', 'Conta[1].acc'] + [f'Reais[{i}]' for i in range(
165   6)]
166
167 for row_idx, tag in enumerate(tags_exibicao, start=1):
168     cor_fundo = "#252526" if row_idx % 2 == 0 else "#2d2d2d"
169
170     lbl_name = tk.Label(frame_tabela, text=tag, font=self.fonte_dados, fg=
171       "#9cdcfe", bg=cor_fundo, anchor="w")
172     lbl_name.grid(row=row_idx, column=0, sticky="nsew", padx=1, pady=1, ipadx=5,
173       ipady=4)
174
175     cor_valores = "#00ffff" if "Reais" in tag else "#ff9900"
176     lbl_value = tk.Label(frame_tabela, text="---", font=self.fonte_lcd_media, fg=
177       cor_valores, bg=cor_fundo, anchor="center")
178     lbl_value.grid(row=row_idx, column=1, sticky="nsew", padx=1, pady=1)
179
180     lbl_status = tk.Label(frame_tabela, text="---", font=self.fonte_dados, fg=
181       "#ce9178", bg=cor_fundo, anchor="center")
182     lbl_status.grid(row=row_idx, column=2, sticky="nsew", padx=1, pady=1)
183
184     self.widgets_dados[tag.lower()] = {
185         "value_widget": lbl_value,
186         "status_widget": lbl_status
187     }
188
189 # -----
190 # PAINEL DO HISTÓRICO GRÁFICO (Canvas Real-Time)
191 # -----
192 frame_grafico = tk.LabelFrame(self.root, text=" TENDÊNCIA EM TEMPO REAL (

```

```

184 Laranja: Conta[0].acc | Ciano: Segundos ) ", font=self.fonte_cabecalho, fg=
185 "#ffffff", bg="#1e1e1e", bd=1, relief=tk.SOLID)
186 frame_grafico.pack(padx=20, pady=10, fill=tk.BOTH, expand=True)
187
188 self.canvas_grafico = tk.Canvas(frame_grafico, bg="#111111", bd=0,
189 highlightthickness=0)
190 self.canvas_grafico.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)
191
192 def atualizar_dados(self):
193     """Efetua a leitura física no CLP via pylogix e redesenha a interface."""
194     retorno = self.comm.Read(TAGS_PARA_LER)
195     sucesso_comunicacao = False
196
197     # Dicionário temporário para processar os valores lidos
198     valores_lidos = {}
199
200     if retorno:
201         sucesso_comunicacao = True
202         for resposta in retorno:
203             if resposta.Status == "Success":
204                 valores_lidos[resposta.TagName.lower()] = resposta.Value
205             else:
206                 sucesso_comunicacao = False
207
208     # Atualiza o LED de Status de Comunicação Geral
209     if sucesso_comunicacao:
210         cor_led = "#00ff00" if self.heartbeat_state else "#00aa00"
211         self.heartbeat_state = not self.heartbeat_state
212     else:
213         cor_led = "#ff0000"
214     self.canvas_led.itemconfig(self.led_id, fill=cor_led)
215
216     # Atualização do Bloco de Horário utilizando os índices do array original
217     try:
218         hora = valores_lidos.get('relogio[3]', 0)
219         minuto = valores_lidos.get('relogio[4]', 0)
220         segundo = valores_lidos.get('relogio[5]', 0)
221         dia = valores_lidos.get('relogio[2]', 1)
222         mes = valores_lidos.get('relogio[1]', 1)
223         ano = valores_lidos.get('relogio[0]', 2026)
224
225         self.lbl_relogio_hora.config(text=f"{hora:02d}:{minuto:02d}:{segundo:02d}")
226         self.lbl_relogio_data.config(text=f"DATA: {dia:02d}/{mes:02d}/{ano:04d}")
227
228         # Alimenta o histórico do gráfico com o valor dos segundos lidos do relógio
229         self.historico_segundos.append(segundo)
230         if len(self.historico_segundos) > self.max_pontos_grafico:
231             self.historico_segundos.pop(0)
232     except Exception:
233         self.lbl_relogio_hora.config(text="--:--:--")
234         self.lbl_relogio_data.config(text="DATA: --/--/----")
235
236     # Atualização da Tabela de Variáveis Dinâmicas
237     for resposta in (retorno if retorno else []):
238         tag_nome_low = resposta.TagName.lower()
239         if tag_nome_low in self.widgets_dados:
240             widgets = self.widgets_dados[tag_nome_low]
241
242             if resposta.Status == "Success":
243                 if isinstance(resposta.Value, float):
244                     texto_valor = f"{resposta.Value:.2f}"
245                 else:
246                     texto_valor = str(resposta.Value)
247
248             widgets["value_widget"].config(text=texto_valor)
249             widgets["status_widget"].config(text="Success", fg="#00ff66")

```

```

248         # Coleta de dados específica para o gráfico (Conta[0].acc)
249         if tag_nome_low == "conta[0].acc":
250             self.historico_conta0.append(resposta.Value)
251             if len(self.historico_conta0) > self.max_pontos_grafico:
252                 self.historico_conta0.pop(0)
253         else:
254             widgets["value_widget"].config(text="---")
255             widgets["status_widget"].config(text="Erro", fg="#ff0000")
256
257     # Caso a comunicação falhe por completo
258     if not sucesso_comunicacao:
259         for tag_low, widgets in self.widgets_dados.items():
260             widgets["value_widget"].config(text="---")
261             widgets["status_widget"].config(text="Offline", fg="#ff0000")
262
263     # Renderiza a atualização do Canvas Gráfico
264     self.desenhar_grafico()
265
266     # Ciclo de atualização contínuo recursivo do Tkinter
267     self.root.after(INTERVALO_ATUALIZACAO, self.atualizar_dados)
268
269     def desenhar_grafico(self):
270         """Desenha as linhas de tendência do histórico coletado dentro do Canvas."""
271         self.canvas_grafico.delete("all")
272
273         largura = self.canvas_grafico.winfo_width()
274         altura = self.canvas_grafico.winfo_height()
275
276         if largura < 10 or altura < 10:
277             return # Evita desenhar se o canvas ainda não foi mapeado na tela
278
279         # Desenha linhas de grade horizontais de referência técnica
280         for i in range(1, 4):
281             y_grade = (altura // 4) * i
282             self.canvas_grafico.create_line(0, y_grade, largura, y_grade, fill="#222222",
283                                             dash=(4, 4))
284
285         def plotar_pena(dados, cor, max_escalas):
286             if len(dados) < 2:
287                 return
288
289             pontos = []
290             fator_x = largura / (self.max_pontos_grafico - 1)
291
292             for idx, valor in enumerate(dados):
293                 x = idx * fator_x
294                 norm_val = max(0, min(valor, max_escalas))
295                 y = altura - ((norm_val / max_escalas) * (altura - 20)) - 10
296                 pontos.append((x, y))
297
298             for i in range(len(pontos) - 1):
299                 self.canvas_grafico.create_line(
300                     pontos[i][0], pontos[i][1],
301                     pontos[i+1][0], pontos[i+1][1],
302                     fill=cor, width=2
303                 )
304
305         # Pena 1: Conta[0].acc (Escala presumida de 0 a 100 para amostragem dinâmica)
306         plotar_pena(self.historico_conta0, "#ff9900", max_escalas=100)
307
308         # Pena 2: Segundos do Relógio (Escala real fixa de 0 a 60)
309         plotar_pena(self.historico_segundos, "#00ffff", max_escalas=60)
310
311     def fechar_aplicativo(self, event=None):
312         """Encerra a conexão de rede e fecha o loop de eventos."""
313         self.comm.Close()
314         self.root.destroy()

```

```
314
315 # =====
316 # INICIALIZAÇÃO DO PROGRAMA
317 # =====
318 if __name__ == "__main__":
319     janela_raiz = tk.Tk()
320     app = MonitorCLPApp(janela_raiz)
321     janela_raiz.mainloop()
322
```